

Security Review Report for Kerne Protocol

July 2026



Table of Contents

1. About Hexens
2. Executive summary
3. Security Review Details
 - Security Review Lead
 - Scope
 - Changelog
4. Severity Structure
 - Severity characteristics
 - Issue symbolic codes
5. Findings Summary
6. Weaknesses
 - Transferable kLP shares can separate withdrawal and esKERNE forfeiture identities
 - esKERNE forfeiture can be bypassed by gas-starving the best-effort forfeit call
 - totalAssets silently under-reports NAV when the verification node call fails
 - _checkCRCircuitBreaker should be applied to captureFounderWealth
 - Unable to update offChainAssets when offChainAssets == 0 and _offChainAssetsBootstrapped == true
 - maxTotalAssets check overestimates deposited assets
 - _checkCRCircuitBreaker should pause the protocol when the collateral ratio exceeds SAFE_CR_THRESHOLD
 - Withdrawal Requests Are Not Bound to the Share Price at Request Time
 - Redundant _checkSolvency() function in KerneVault
 - Redundant extracted variable in captureFounderWealth function

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This audit covers Kerne, a delta-neutral synthetic dollar protocol on the Base network that leverages LST collateral and on-chain perpetuals hedging on Hyperliquid to deliver capital-efficient, delta-neutral yield backed by reserves that are verifiable on-chain.

The review was conducted over a one-week period and included a comprehensive analysis of the relevant smart contracts.

During the assessment, we identified two high-severity issues that could allow users to bypass the `esKERNE` forfeiture mechanism by transferring shares prior to withdrawal or by exploiting gas starvation. Additionally, we identified two medium-severity issues, four low-severity issues, and two informational findings.

All identified issues were either remediated or acknowledged by the development team and subsequently verified by our auditors.

Following the remediation phase, we conclude that the protocol's overall security posture and code quality have improved as a result of this audit.

3. Security Review Details

- **Review Led by**

Trung Dinh, Lead Security Researcher

- **Scope**

The analyzed resources are located on:

<https://github.com/kerne-protocol/contracts-public/tree/58a838dd749c63206fcd5c6f1bc275af77728805/audits/scope/0912c870/src>

The issues described in this report were fixed in the following commit:

<https://github.com/kerne-protocol/contracts-public/tree/58a838dd749c63206fcd5c6f1bc275af77728805/audits/scope/98f29e55/src>

- **Changelog**

■ 13 July 2026	Audit start
■ 20 July 2026	Initial report
■ 21 July 2026	Revision received
■ 31 July 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

- 1. Impact of the vulnerability
- 2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

▪ Severity Characteristics

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical	Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.
High	Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium

Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.

Low

Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.

Informational

Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

▪ Issue Symbolic Codes

Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

Severity

Number of findings

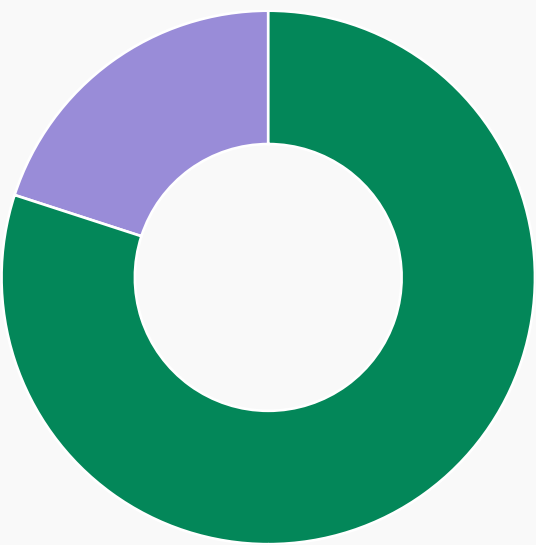
Critical	0
High	2
Medium	2
Low	4
Informational	2

Total:

10



- High
- Medium
- Low
- Informational



- Fixed
- Acknowledged

6. Weaknesses

This section contains the list of discovered weaknesses.

KERNE1-4 | Transferable kLP shares can separate withdrawal and esKERNE forfeiture identities

Acknowledged

Severity:

High

Probability:

Likely

Impact:

High

Path:

src/KerneVault.sol#L1605-L1622

Description:

KerneVault triggers esKERNE forfeiture only when a withdrawal request leaves `msg.sender` with less than the configured share threshold. The same address is then passed to `esKERNE.forfeit()`.

```
address esc = escrowToken;
if (esc != address(0)) {
    uint256 residualShares = balanceOf(msg.sender);
    uint256 dustFloor = 10 ** _decimalsOffset();
    uint256 peakFloor = (forfeitureHighWaterShares[msg.sender] * FORFEIT_EXIT_RETAIN_BPS) / 10000;
    uint256 exitFloor = peakFloor > dustFloor ? peakFloor : dustFloor;
    if (residualShares < exitFloor) {
        try IEscrowForfeiter(esc).forfeit(msg.sender) {
            emit EscrowForfeitTriggered(msg.sender);
            // Re-anchor the peak to the post-exit residual so a subsequent re-deposit begins
            // a fresh high-water and partial withdrawals from the new position are not
            // measured against the pre-exit peak.
            forfeitureHighWaterShares[msg.sender] = residualShares;
        } catch {
            // Best-effort: never let a forfeiture failure brick a withdrawal.
        }
    }
}
```

```

function forfeit(
    address user
) external onlyRole(VAULT_ROLE) updateReward(user) {
    uint256 unvestedAmount = unvested(user);
    if (unvestedAmount == 0) return;

    // Remove unvested from user's balance
    balanceOf[user] -= unvestedAmount;
    totalSupply -= unvestedAmount;
    totalForfeited += unvestedAmount;

    uint256 remainingSupply = totalSupply - balanceOf[user];
    if (remainingSupply > 0) {
        rewardPerTokenStored += (unvestedAmount * 1e18) / remainingSupply;
        // Do NOT re-add to totalSupply here. claimRedistribution() will
        // add to both balanceOf[claimant] and totalSupply on claim.
    }

    // If user has zero balance remaining, reset their vesting state entirely.
    if (balanceOf[user] == 0) {
        vestingStart[user] = 0;
        claimed[user] = 0;
    } else {
        vestingStart[user] = block.timestamp > VESTING_DURATION ? block.timestamp - VESTING_DURATION
: 1;
        userRewardPerTokenPaid[user] = rewardPerTokenStored;
    }

    emit Forfeited(user, unvestedAmount, remainingSupply > 0 ? unvestedAmount : 0);
}

```

Vault shares remain transferable through the inherited ERC-20 implementation, while **esKERNE** balances and vesting state are maintained separately for each address. A depositor who has received unvested **esKERNE** can therefore transfer all **kLP** shares to another address. The recipient can request and claim the withdrawal, causing forfeiture to be evaluated against the recipient, which has no **esKERNE** balance. The original depositor retains the unvested **esKERNE** and may convert it after vesting if the conversion reserve is funded. This allows the economic position to exit without the associated unvested rewards being forfeited and redistributed to the remaining holders.

Remediation:

- Bind **esKERNE** forfeiture eligibility to changes in **kLP** ownership rather than only to the withdrawal caller.
- Make kLP shares non-transferable, or reconcile the sender's unvested **esKERNE** when shares are transferred.
- Apply the same position-to-reward invariant across **transfer**, **transferFrom**, withdrawal requests, and conversion.

Commentary from the client:

“Currently esKERNE totalSupply() and totalEmitted() are both 0, so forfeit() has nothing to act on, and that esKERNE's kerneToken points at the retired v1 token, so conversion cannot pay out either. We will design the ownership-to-forfeiture binding properly and re-test it before enabling forfeiture.”

KERNE1-5 | esKERNE forfeiture can be bypassed by gas-starving the best-effort forfeit call

Fixed ✓

Severity:

High

Probability:

Likely

Impact:

High

Path:

src/KerneVault.sol#L1612-L1620

Description:

The function `KerneVault.requestWithdrawal()` is used to queue a withdrawal request for a given asset amount. If the user makes a full economic exit from the vault, the function will trigger the forfeiture on the **esKERNE** contract. The trigger is invoked as a best-effort external call wrapped in a `try/catch` at the very end of `requestWithdrawal()`.

```
function requestWithdrawal(
    uint256 assets
) external nonReentrant whenNotPaused returns (uint256) {
    ...

    if (residualShares < exitFloor) {
        try IEscrowForfeiter(esc).forfeit(msg.sender) {
            emit EscrowForfeitTriggered(msg.sender);
            // Re-anchor the peak to the post-exit residual so a subsequent re-deposit begins
            // a fresh high-water and partial withdrawals from the new position are not
            // measured against the pre-exit peak.
            forfeitureHighWaterShares[msg.sender] = residualShares;
        } catch {
            // Best-effort: never let a forfeiture failure brick a withdrawal.
        }
    }
    return requestId;
}
```

Because of the EVM's EIP-150 "all but one 64th" gas-forwarding rule, an exiting user can set the transaction gas limit so that the `forfeit()` sub-call runs out of gas and reverts, while the parent `requestWithdrawal()` frame retains the ~1/64 of gas it needs to finish. The `catch` swallows the failure, the withdrawal request is recorded, and the user keeps 100% of their unvested esKERNE.

Feasibility

Let:

- **G** = gas at function entry (the attacker sets this via the tx gas limit).
- **P** = the deterministic gas cost of everything before the **forfeit** call.

The gas forwarded to **forfeit** is:

```
forwarded ≈ 63/64 · (G - P)
retained  ≈ 1/64 · (G - P)
```

With **F** = gas **forfeit()** needs to complete and **R** = gas for the tail after the **catch**, the attack succeeds when the attacker chooses **G** such that

$$64 \cdot R \leq G - P < (64/63) \cdot F$$

- Lower bound **64·R**: guarantees the retained **(G-P)/64** covers the tail so the parent transaction still succeeds and the withdrawal is persisted.
- Upper bound **(64/63)·F**: guarantees the forwarded **63/64·(G-P)** is below what **forfeit()** needs, so it runs out of gas and reverts into the **catch**.

Because **R** is a few hundred gas and **F** is tens of thousands, the condition **F > 63·R** holds by a wide margin, so the window is large. **P** is deterministic for a given state, so the attacker can simulate off-chain (**eth_call** / local fork), computes the exact gas limit, and lands **G** on the first attempt.

Remediation:

Consider requiring a minimum amount of gas to be available immediately before invoking **forfeit()**, such that the gas forwarded under EIP-150 (**63/64** of the remaining gas) is sufficient for the call to complete successfully.

```
if (gasleft() < MIN_FORFEIT_GAS) revert InsufficientGasForForfeit();
try IEscrowForfeiter(esc).forfeit(msg.sender) { ... } catch { ... }
```

Optionally, an explicit gas stipend can be forwarded:

```
try IEscrowForfeiter(esc).forfeit{gas: MIN_FORFEIT_GAS}(msg.sender) { ... } catch { ... }
```

KERNE1-7 | totalAssets silently under-reports NAV when the verification node call fails

Fixed ✓

Severity:

Medium

Probability:

Rare

Impact:

High

Path:

src/KerneVault.sol#L878

Description:

When a **verificationNode** is configured, **totalAssets()** queries it via **staticcall** to incorporate the vault's off-chain / L1 assets (Proof-of-Reserve). If the call reverts or returns anything other than exactly 32 bytes, the function silently falls back to returning only the on-chain tracked assets and prime allocations, completely excluding the off-chain portion of the vault's NAV.

```
function totalAssets() public view virtual override returns (uint256) {
    address node = verificationNode;
    if (node != address(0)) {
        (bool success, bytes memory data) =
            node.staticcall(abi.encodeWithSignature("getVerifiedAssets(address)", address(this)));

        if (success && data.length == 32) {
            return _trackedOnChainAssets + totalPrimeAllocated + abi.decode(data, (uint256));
        }

        return _trackedOnChainAssets + totalPrimeAllocated;
    }
    return _trackedOnChainAssets + offChainAssets + l1Assets + hedgingReserve + totalPrimeAllocated;
}
```

Notably, the fallback path does not reuse the strategist-maintained asset buckets (**offChainAssets + l1Assets + hedgingReserve**) that are used when no verification node is configured. As a result, a vault with a configured verification node can report fewer assets than a vault without one whenever the node is temporarily unavailable.

Since **totalAssets()** is used for share pricing, solvency checks, and deposit cap enforcement, this silent under-reporting affects the entire vault during a node failure window.

For example, if the verification node fails, the reported NAV may become significantly lower than the vault's actual value. Consequently, the share price is understated, allowing users to mint more

shares than they should receive for their deposits. Once the verification node recovers and the correct NAV is reported again, those users can redeem their shares at the higher share price and extract value from existing vault holders.

Remediation:

On node failure, consider returning the no-node formula **_trackedOnChainAssets + offChainAssets + liAssets + hedgingReserve + totalPrimeAllocated** instead of dropping the off-chain buckets.

KERNE1-9 | `_checkCRCircuitBreaker` should be applied to `captureFounderWealth`

Fixed 

Severity:

Medium

Probability:

Unlikely

Impact:

Medium

Path:

`src/KerneVault.sol#L1303-L1372`

Description:

In the `KerneVault` contract, the `_checkCRCircuitBreaker()` function is used to evaluate the vault's live collateral ratio against predefined thresholds and update the Red Halt solvency circuit breaker state accordingly.

```
// --- CR Circuit Breaker ---
function _checkCRCircuitBreaker() internal {
    uint256 cr = getSolvencyRatio();
    if (cr < CRITICAL_CR_THRESHOLD) {
        if (!crCircuitBreakerActive) {
            crCircuitBreakerActive = true;
            crCircuitBreakerTriggeredAt = block.timestamp;
            emit CRCircuitBreakerTriggered(cr, block.timestamp);
            if (!paused()) _pause();
        }
    } else if (cr < WARNING_CR_THRESHOLD) {
        if (!crSoftAlertActive) {
            crSoftAlertActive = true;
            emit CRSoftAlertTriggered(cr, block.timestamp);
        }
    } else if (cr >= SAFE_CR_THRESHOLD) {
        if (crCircuitBreakerActive && block.timestamp >= crCircuitBreakerTriggeredAt +
            crCircuitBreakerCooldown) {
            crCircuitBreakerActive = false;
            emit CRCircuitBreakerRecovered(cr, block.timestamp);
        }
        if (crSoftAlertActive) {
            crSoftAlertActive = false;
            emit CRSoftAlertRecovered(cr, block.timestamp);
        }
    }
}
```

The Red Halt mechanism is intended to pause the protocol whenever the collateral ratio falls below **CRITICAL_CR_THRESHOLD** (9,900 bps = 99%), preventing users from interacting with a stale or incorrect share price during an insolvency event.

Currently, **_checkCRCircuitBreaker()** is invoked by **updateOffChainAssets()**, **updateHedgingReserve()**, and **updateL1Assets()** because these functions can directly reduce **totalAssets()**, and the protocol should react immediately when the collateral ratio becomes unsafe.

However, **captureFounderWealth()** can also reduce **totalAssets()** by decreasing **_trackedOnChainAssets**, yet it does not invoke **_checkCRCircuitBreaker()**.

```
function captureFounderWealth(  
    uint256 grossYieldAmount  
) external onlyRole(STRATEGIST_ROLE) {  
  
    if (rewardReserveAmount > 0) {  
        ...  
        _trackedOnChainAssets -= rewardReserveAmount;  
        ...  
    }  
  
    if (founderRevenueAmount > 0) {  
        ...  
        _trackedOnChainAssets -= founderRevenueAmount;  
        ...  
    }  
  
    // @audit no _checkCRCircuitBreaker() is invoked  
}
```

Since **captureFounderWealth()** can reduce **totalAssets()** by up to 5%, it may cause the collateral ratio to fall below the critical threshold without triggering the circuit breaker. As a result, the protocol remains operational and users may continue to interact with mispriced shares.

Remediation:

Invoke **_checkCRCircuitBreaker()** at the end of **captureFounderWealth()** to ensure the Red Halt mechanism is consistently applied whenever **totalAssets()** decreases significantly.

KERNE1-2 | Unable to update offChainAssets when offChainAssets == 0 and _offChainAssetsBootstrapped == true

Fixed 

Severity:

Low

Probability:

Rare

Impact:

Low

Path:

```
src/KerneVault.sol#L966-L987  
src/KerneVault.sol#L1008-L1025  
src/KerneVault.sol#L1043-L1060
```

Description:

Function `KerneVault.updateOffChainAssets()` is used to update the `offChainAssets` value.

```
function updateOffChainAssets(  
    uint256 amount  
) external onlyRole(STRATEGIST_ROLE) {  
    ...  
  
    uint256 oldAmount = offChainAssets;  
    if (_offChainAssetsBootstrapped && maxOffChainChangeRateBps > 0) {  
        // Once bootstrapped, oldAmount==0 still applies the rate-limit math  
        // by treating the change against a small floor (1 wei). This is what  
        // closes the original bypass: a zeroed-then-rewritten path now fails  
        // the rate-limit check the same way any other write would.  
        uint256 baseline = oldAmount > 0 ? oldAmount : 1;  
        uint256 maxChange = (baseline * maxOffChainChangeRateBps) / 10000;  
        uint256 change = amount > oldAmount ? amount - oldAmount : oldAmount - amount;  
        if (change > maxChange) revert OffChainChangeExceedsMaxRate();  
    }  
  
    ...  
}
```

Once `_offChainAssetsBootstrapped` is enabled, the rate-limit check is applied even when `oldAmount` is `0`. In this case, the code uses a fallback `baseline` value of `1` to prevent the previously identified rate-limit bypass.

However, this introduces a new issue. The maximum value of `maxOffChainChangeRateBps` is **5000** (50%), which is always less than **10000**. Therefore, when `oldAmount == 0`:

```
uint256 baseline = 1;  
uint256 maxChange = (1 * 5000) / 10000; // = 0
```

As a result, `maxChange` is always **0**. Any attempt to update `offChainAssets` from **0** to a non-zero value produces a non-zero `change`, causing the following check to always revert:

```
if (change > maxChange) revert OffChainChangeExceedsMaxRate();
```

Consequently, once `offChainAssets` reaches **0** while `_offChainAssetsBootstrapped == true`, it becomes impossible to set `offChainAssets` back to a non-zero value. The only recovery path is calling `resetBucketBootstrap()` to reset `_offChainAssetsBootstrapped` to `false`.

Note that the issue also applies to the function `updateHedgingReserve()` and `updateL1Assets()` as well.

Remediation:

Consider handling the `oldAmount == 0` case explicitly. For example, revert with a dedicated error indicating that the bootstrap state must be reset before updating `offChainAssets`.

KERNE1-3 | maxTotalAssets check overestimates deposited assets

Fixed 

Severity:

Low

Probability:

Rare

Impact:

Low

Path:

src/KerneVault.sol#L1971

Description:

Function **KerneVault._deposit()** is used to deposit assets into the vault. Before processing the deposit, it verifies that the vault's total assets do not exceed the configured cap.

```
function _deposit(
    address caller,
    address receiver,
    uint256 assets,
    uint256 shares
) internal override {
    ...

    if (maxTotalAssets > 0 && totalAssets() + assets > maxTotalAssets) revert DepositCapExceeded();

    _trackedOnChainAssets += assets - fee;

    ...
}
```

The check is incorrect because it assumes the entire **assets** amount will be added to the vault. In reality, a portion of the deposit is deducted as **fee** and transferred to the treasury/founder, while only **assets - fee** is added to **_trackedOnChainAssets** and contributes to the vault's total assets.

As a result, the check can incorrectly revert deposits that would leave the vault below **maxTotalAssets**, unnecessarily preventing valid deposits.

Remediation:

Consider excluding the deposit fee when enforcing the **maxTotalAssets** limit, since only the net deposited amount is added to the vault.


```
- if (maxTotalAssets > 0 && totalAssets() + assets > maxTotalAssets) revert DepositCapExceeded();  
+ if (maxTotalAssets > 0 && totalAssets() + assets - fee > maxTotalAssets) revert DepositCapExceeded();
```

KERNE1-10 | `_checkCRCircuitBreaker` should pause the protocol when the collateral ratio exceeds `SAFE_CR_THRESHOLD`

Acknowledged

Severity:

Low

Probability:

Rare

Impact:

Medium

Path:

src/KerneVault.sol#L2126-L2151

Description:

The `_checkCRCircuitBreaker()` function is used to evaluate the vault's live collateral ratio against predefined thresholds and update the Red Halt solvency circuit breaker state accordingly.

```
// --- CR Circuit Breaker ---
function _checkCRCircuitBreaker() internal {
    uint256 cr = getSolvencyRatio();
    if (cr < CRITICAL_CR_THRESHOLD) {
        if (!crCircuitBreakerActive) {
            crCircuitBreakerActive = true;
            crCircuitBreakerTriggeredAt = block.timestamp;
            emit CRCircuitBreakerTriggered(cr, block.timestamp);
            if (!paused()) _pause();
        }
    }
    else if (cr < WARNING_CR_THRESHOLD) {
        if (!crSoftAlertActive) {
            crSoftAlertActive = true;
            emit CRSoftAlertTriggered(cr, block.timestamp);
        }
    }
    else if (cr >= SAFE_CR_THRESHOLD) {
        if (crCircuitBreakerActive && block.timestamp >= crCircuitBreakerTriggeredAt +
crCircuitBreakerCooldown) {
            crCircuitBreakerActive = false;
            emit CRCircuitBreakerRecovered(cr, block.timestamp);
        }
        if (crSoftAlertActive) {
            crSoftAlertActive = false;
            emit CRSoftAlertRecovered(cr, block.timestamp);
        }
    }
}
```

```

    }
  }
}

```

Currently, the function pauses the protocol immediately when **cr** is below **CRITICAL_CR_THRESHOLD** (99%). In contrast, when **cr** is greater than or equal to **SAFE_CR_THRESHOLD** (101%), it clears **crCircuitBreakerActive** and **crSoftAlertActive**, treating the vault as healthy.

Under normal circumstances, this behavior is reasonable because the vault is overcollateralized. However, if the strategist becomes malicious (which is explicitly considered in the comments around line 994), an artificially inflated collateral ratio can be exploited to extract value from the vault.

For example, the **updateL1Assets()** function, which is restricted to the strategist, allows updating **l1Assets** within the **maxOffChainChangeRateBps** threshold:

```

function updateL1Assets(
  uint256 amount
) external onlyRole(STRATEGIST_ROLE) {
  if (block.timestamp < lastL1AssetsTimestamp + offChainUpdateCooldown) revert UpdateCooldownNotMet();
  uint256 oldAmount = l1Assets;
  if (_l1AssetsBootstrapped && maxOffChainChangeRateBps > 0) {
    uint256 baseline = oldAmount > 0 ? oldAmount : 1;
    uint256 maxChange = (baseline * maxOffChainChangeRateBps) / 10000;
    uint256 change = amount > oldAmount ? amount - oldAmount : oldAmount - amount;
    if (change > maxChange) revert OffChainChangeExceedsMaxRate();
  }
  l1Assets = amount;
  if (amount > 0) _l1AssetsBootstrapped = true;
  lastL1AssetsTimestamp = block.timestamp;
  lastReportedTimestamp = block.timestamp;
  emit L1AssetsUpdated(oldAmount, amount, block.timestamp);
  _checkCRCircuitBreaker();
}

```

In the best case, the strategist can increase **l1Assets** by **DEFAULT_MAX_OFFCHAIN_CHANGE_RATE_BPS** (20%), which correspondingly increases **totalAssets()** and the vault's share price. Since an increase in the collateral ratio does not trigger a pause, users can continue interacting with the vault using the inflated share price.

For example:

1. The strategist deposits assets into the vault and receives **X** shares.
2. The strategist calls **updateL1Assets()** to artificially increase **totalAssets()**, thereby increasing the share price.
3. The strategist requests and claims the withdrawal of **X** shares, profiting from the difference in share price before and after the **updateL1Assets()** call.

Remediation:

Consider pausing the protocol when the collateral ratio exceeds a predefined upper threshold (e.g., **SAFE_CR_THRESHOLD**). This provides an additional safeguard against malicious or incorrect off-chain asset reporting by preventing users from interacting with the vault while the reported collateral ratio is unexpectedly high.

Commentary from the client:

*“A healthy vault runs above **SAFE_CR_THRESHOLD**, so a hard pause there would brick normal operation, and the **KERNE1-11** fix removes the profit from the scenario it describes.”*

KERNE1-11 | Withdrawal Requests Are Not Bound to the Share Price at Request Time

Fixed 

Severity:

Low

Probability:

Rare

Impact:

Medium

Path:

src/KerneVault.sol#L1656

Description:

In the **KerneVault** contract, the **claimWithdrawal()** function is used to claim a matured withdrawal request and receive the underlying assets. Currently, the function calculates the amount of assets to return using the current share price and ignores the share price at the time the withdrawal request was created.

```
function claimWithdrawal(  
    uint256 requestId  
) external nonReentrant whenNotPaused {  
    ...  
    uint256 assetsOut = convertToAssets(req.shares);  
    ...  
}
```

This introduces an issue where users can queue their shares for withdrawal and, after the `unlockTimestamp`, selectively claim them based on favorable changes in the vault's NAV. In particular, users can monitor events that reduce `totalAssets()` and front-run other users to avoid realizing those losses.

For example:

1. Setup:
 - `totalAssets() = 200`
 - `_trackedOnChainAssets = 100`
 - `l1Assets = 100`
 - `offChainAssets = hedgingReserve = totalPrimeAllocated = 0`
 - `totalSupply() = 200`
2. Alice deposits 100 assets into the vault and receives 100 shares (fees are ignored in this example).
 - `totalAssets() = 200 + 100 = 300`
 - `_trackedOnChainAssets = 100 + 100 = 200`

- `l1Assets = 100`
 - `totalSupply() = 200 + 100 = 300`
3. Alice immediately queues all 100 shares for withdrawal. After the request's **`unlockTimestamp`**, she can call **`claimWithdrawal()`** at any time.
 4. Assume the L1 strategy generates a profit of 20 assets, and the strategist calls **`updateL1Assets()`**:
 - `l1Assets = 100 + 20 = 120`
 - `totalAssets() = 200 + 120 = 320`
 5. After some time, the L1 strategy incurs a loss of 20 assets, and the strategist calls **`updateL1Assets()`** to reduce **`l1Assets`** by 20.
 6. Alice observes the upcoming loss and front-runs the event by calling **`claimWithdrawal()`** before the share price decreases. She receives:

```
convertToAssets(100) = 100 * 320 / 300 = 106
```

By doing so, Alice avoids the 6-asset loss that should have been borne by her shares.

As shown above, once the **`unlockTimestamp`** has passed, users can claim their withdrawal using the current share price and effectively avoid negative NAV changes arising from **`l1Assets`**, **`offChainAssets`**, or **`hedgingReserve`**.

Remediation:

By the time a user requests a withdrawal, they should forfeit any future yield generated by the vault. Consider capping the withdrawal amount at the lesser of the current share value and the asset amount recorded when the withdrawal request was created.

```
-- uint256 assetsOut = convertToAssets(req.shares);
++ uint256 assetsOut = min(convertToAssets(req.shares), req.assets);
```

If **`convertToAssets(req.shares)`** is greater than **`req.assets`**, the excess yield can either be transferred to the treasury/founder or distributed among the remaining shareholders.

Furthermore, consider adding a **`cancelWithdrawal()`** function that allows user cancel a specific request.

KERNE1-6 | Redundant `_checkSolvency()` function in KerneVault

Fixed 

Severity:

Informational

Probability:

Rare

Impact:

Informational

Path:

src/KerneVault.sol#L903-L907

Description:

The **KerneVault** contract defines the internal function `_checkSolvency()`. However, the function is never used anywhere in the codebase.

```
function _checkSolvency(  
    bool strict  
) internal {  
    _updateSolvency(strict);  
}
```

Remediation:

Consider removing the function if it is not intended to be used. This improves code clarity and reduces unnecessary code.

KERNE1-8 | Redundant extracted variable in captureFounderWealth function

Fixed 

Severity:

Informational

Probability:

Rare

Impact:

Informational

Path:

src/KerneVault.sol#L1347

Description:

In the function `KerneVault.captureFounderWealth()`, line 1347 calculates the variable **extracted** as:

```
uint256 extracted = rewardReserveAmount + founderRevenueAmount;
```

However, **founderRevenueAmount** is calculated as follows on line 1336:

```
uint256 founderRevenueAmount = totalFee - rewardReserveAmount;
```

This is equivalent to:

```
totalFee = founderRevenueAmount + rewardReserveAmount
```

Therefore, **extracted** is always equal to **totalFee**, making the intermediate variable redundant.

Remediation:

Consider removing **extracted** and using **totalFee** directly instead of **extracted** in the function if it is not intended to be used. This improves code clarity and reduces unnecessary code.

hexens x  Kerne

